TALLER SEMANA #10

INTEGRANTES:

Jhorger David Andrade Calderon

Cesar David Ojeda Gonzalez

PROFESOR:

Eliecer Montero Ojeda

PATRONES DE SOFTWARE

GRUPO: E196

UNIDADES TECNOLOGICAS DE SANTANDER

BUCARAMANGA

2026

TABLA DE CONTENIDO

TABLA DE CONTENIDO 2
INTRODUCCIÓN 3
DESCRIPCIÓN DEL PROYECTO 4
OBJETIVOS 4
General 4
Específicos 4
DIAGRAMA GENERAL 5
ARQUITECTURA HEXAGONAL 6
ESTRUCTURA DEL PROYECTO 6
PATRÓN SINGLETON (ENUM) 8
PATRÓN FACTORY METHOD 10
PATRÓN ABSTRACT FACTORY 16
PATRÓN BUILDER (FLUENT BUILDER) 20
PATRÓN PROTOTYPE 23
PATRÓN ADAPTER 27
PATRÓN BRIDGE 29
PATRÓN DECORATOR 33
PRUEBAS DE FUNCIONALIDAD DEL PROYECTO 37
REFERENCIAS BIBLIOGRÁFICAS 42

INTRODUCCIÓN

La transformación digital en el ámbito educativo ha impulsado la necesidad de contar con plataformas tecnológicas que faciliten el acceso a recursos académicos de manera rápida, organizada y accesible desde cualquier lugar. En este contexto, las bibliotecas digitales se han convertido en una herramienta fundamental para estudiantes, docentes e investigadores, ya que permiten consultar y gestionar contenido bibliográfico sin las limitaciones de una biblioteca física, optimizando así los procesos de aprendizaje y acceso a la información.

El presente documento describe el desarrollo de un Sistema de Biblioteca Digital, una plataforma diseñada para gestionar de manera eficiente libros electrónicos, préstamos, devoluciones y reservas dentro de un entorno digital. El sistema permitirá organizar el catálogo de libros y ofrecer acceso a diferentes formatos de lectura como PDF, EPUB y MOBI, facilitando su consulta desde distintos dispositivos. Asimismo, la plataforma incorporará un modelo de suscripciones y un módulo de recomendaciones que mejorará la experiencia del usuario mediante sugerencias basadas en sus intereses y comportamiento de lectura.

Además, el proyecto contempla la aplicación de principios de arquitectura moderna y buenas prácticas de desarrollo de software con el objetivo de construir un sistema escalable, mantenible y capaz de soportar acceso simultáneo de múltiples usuarios. Asimismo, durante su desarrollo se incorporará la aplicación de patrones de diseño (GoF), de esta manera, el sistema busca ofrecer una solución tecnológica robusta que contribuya a la gestión eficiente de recursos bibliográficos en entornos educativos y académicos.

DESCRIPCIÓN DEL PROYECTO

El Sistema de Biblioteca Digital es una plataforma tecnológica orientada al ámbito educativo y académico, diseñada para facilitar el acceso, la gestión y la distribución de contenido bibliográfico digital de forma eficiente, segura y escalable. Su objetivo es centralizar la administración de recursos digitales y ofrecer a estudiantes, docentes e investigadores un entorno que permita consultar, prestar y gestionar libros electrónicos desde cualquier dispositivo con conexión a internet. La plataforma permitirá administrar el catálogo digital mediante el registro, clasificación y organización de libros en diferentes áreas del conocimiento, además de gestionar procesos de préstamos, devoluciones y reservas.

El sistema será compatible con múltiples formatos de lectura como PDF, EPUB y MOBI, lo que permitirá a los usuarios acceder al contenido desde distintos dispositivos y aplicaciones de lectura. Asimismo, incorporará un modelo de suscripciones para gestionar diferentes niveles de acceso y beneficios, junto con un módulo de recomendaciones personalizadas que sugerirá libros según el comportamiento y las preferencias de cada usuario.

Desde el punto de vista técnico, el sistema estará diseñado para soportar acceso multiusuario simultáneo, aplicando principios de arquitectura moderna y buenas prácticas de desarrollo de software que garanticen un alto nivel de rendimiento, escalabilidad y mantenibilidad de la plataforma.

OBJETIVOS

General

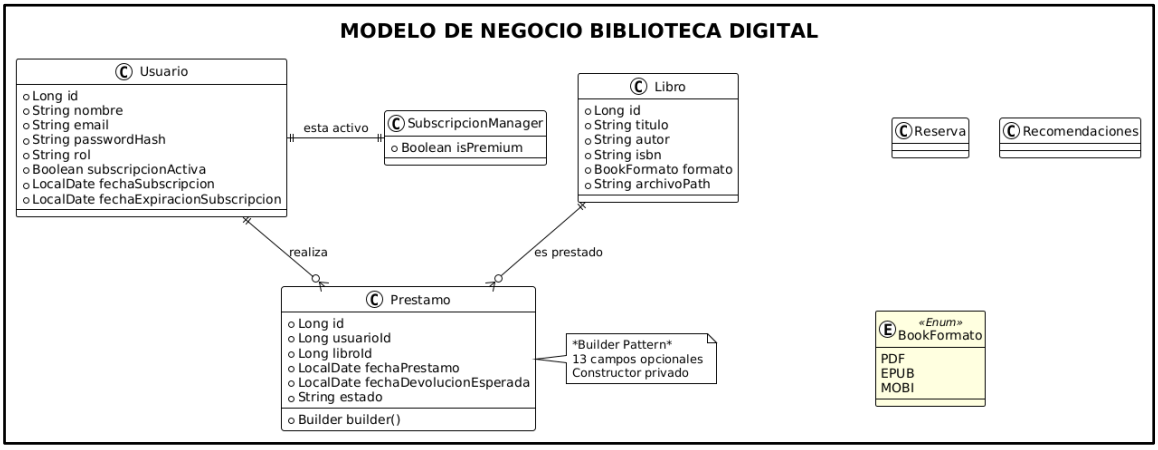
Diseñar e implementar un Sistema de Biblioteca Digital que permita gestionar de manera eficiente el catálogo de libros electrónicos, los procesos de préstamos, devoluciones y reservas, así como un modelo de suscripciones y recomendaciones, garantizando acceso multiusuario simultáneo, compatibilidad con formatos como PDF, EPUB y MOBI, y asegurando escalabilidad y rendimiento.

Específicos

- Diseñar un sistema para la gestión de libros digitales que permita almacenar, consultar y administrar el contenido de la biblioteca.
- Implementar funcionalidades para administrar préstamos, devoluciones y reservas de libros dentro del sistema.
- Desarrollar un sistema de suscripciones que permita gestionar distintos tipos de acceso y beneficios para los usuarios.
- Aplicar mecanismos que permitan el acceso simultáneo multiusuario al sistema sin afectar la integridad de la información.

DIAGRAMA GENERAL

El modelo de negocio propuesto para la biblioteca digital plantea una plataforma en la que el usuario se convierte en el centro de todas las operaciones futuras del sistema. A partir de la gestión de usuarios, libros y préstamos ya implementada, se proyecta incorporar componentes adicionales como la gestión de suscripciones, reservas y recomendaciones, que permitirán ofrecer experiencias más personalizadas y controladas. Este diseño anticipa cómo cada usuario, según su tipo de suscripción y su historial de préstamos, podrá acceder a diferentes catálogos de libros, reservar títulos específicos y recibir sugerencias automáticas.



ARQUITECTURA HEXAGONAL

El proyecto de la Biblioteca Digital está organizado siguiendo el modelo de arquitectura hexagonal o Ports and Adapters, el cual permite separar claramente la lógica de negocio del código dependiente de frameworks y tecnologías externas. La aplicación se divide en tres capas principales: Domain, Application e Infrastructure. El dominio representa el núcleo del sistema y las demás capas dependen de él, garantizando que la lógica de negocio permanezca aislada de detalles técnicos como bases de datos, frameworks o protocolos de comunicación.

Capa de Dominio (Domain)

La capa domain contiene el modelo de negocio y la lógica central del sistema. Dentro de esta se encuentran las carpetas model, donde se definen las entidades principales del sistema; port.in, que contiene los puertos de entrada o contratos de los casos de uso; y port.out, donde se definen los puertos de salida utilizados para la persistencia de datos.

También incluye la carpeta service, donde se ubican servicios de dominio como la gestión de subida de archivos, organizada en subcarpetas como upload.factory, upload.factory.creators y upload.uploaders, que implementan la lógica para manejar diferentes formatos de archivos. Además, existe la carpeta subscription, encargada de la gestión relacionada con suscripciones dentro del sistema.

Capa de Aplicación (Application)

La capa application contiene los servicios de aplicación responsables de orquestar los casos de uso del sistema. En esta capa se encuentra la carpeta service, donde se ubican los servicios que coordinan la interacción entre los puertos definidos en el dominio y los adaptadores de infraestructura, como BookService, PrestamoService, RecomendacionService, SubscriptionService y UserService.

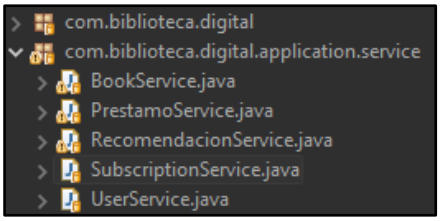
Capa de Infraestructura (Infrastructure)

La capa infrastructure contiene los adaptadores que conectan el sistema con tecnologías externas. Dentro de adapter.in.web se encuentran los controladores que exponen la API REST, como BookController, PrestamoController, RecomendacionesController y UserController. Por otro lado, adapter.out.persistence contiene la integración con la base de datos, organizada en las subcarpetas adapter, entity y repository. Finalmente, la carpeta config incluye configuraciones generales de la aplicación, como BeanConfiguration y GlobalExceptionHandler.

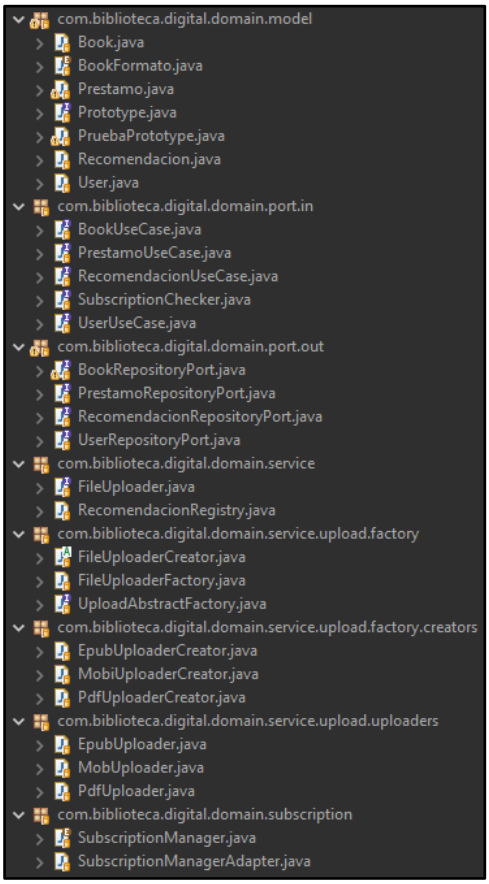
ESTRUCTURA DEL PROYECTO

Después de implementar los diferentes patrones de diseño estudiados durante el corte, se definió la estructura final del proyecto. Esta organización se basa en la arquitectura hexagonal, lo que permite separar adecuadamente las responsabilidades entre las distintas capas del sistema. Dentro de esta estructura se incluyen las clases principales del dominio, como Usuario, Libro y Préstamo, junto con las carpetas correspondientes que soportan su funcionamiento y facilitan la organización.

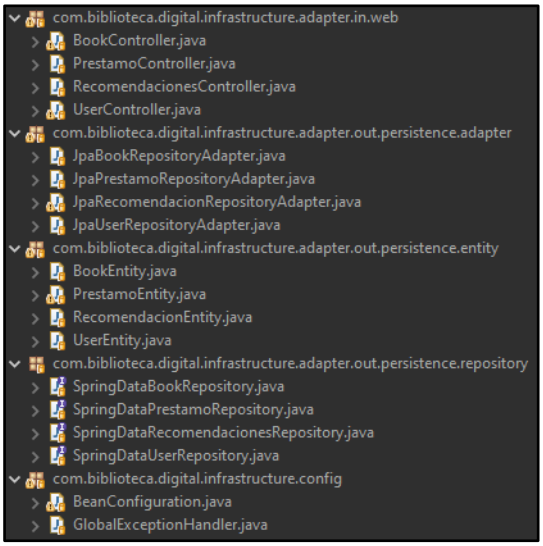
Application



Domain



Infrastrucure



PATRÓN SINGLETON (ENUM)

El Singleton es un patrón de diseño creacional que garantiza que una clase tenga una única instancia y proporcione un punto de acceso global a la misma, tal como lo describen GeeksforGeeks y Refactoring.Guru al enfatizar la “single instance” y el “global access point”. Según GeeksforGeeks, este patrón se utiliza cuando se necesita control centralizado sobre recursos compartidos como configuraciones, logging o conexiones a base de datos, donde múltiples instancias serían costosas o inconsistentes. Refactoring.Guru y Amit Bhardwaj coinciden en que su intención principal es controlar el acceso a un recurso compartido y ofrecer un punto de acceso global bien definido, lo que explica sus distintas variantes:

- ✓ Eager Initialization
- ✓ Lazy Initialization (thread-safe)
- ✓ Double-Checked Locking
- ✓ Enum Singleton (recomendado por Bloch).

- **Código implementado**

*SubscriptionManager.java

```java
package com.biblioteca.digital.domain.subscription;

import java.time.LocalDate;

public enum SubscriptionManager {

    INSTANCE;

    public boolean isPremium(User user) {

        System.out.println("HashCode instancia: " + this.hashCode());

        if (!user.isSuscripcionActiva()) {
            return false;
        }

        if (user.getFechaExpiracionSuscripcion() == null) {
            return false;
        }

        return user.getFechaExpiracionSuscripcion().isAfter(LocalDate.now());
    }
}
```

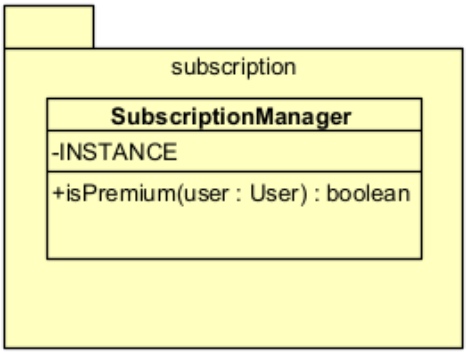
- **Justificación de la implementación**

Se implementó el patrón Singleton basado en enum. Amit Bhardwaj destaca que el Enum Singleton en Java es una de las formas más simples y seguras de implementar un Singleton, ya que la JVM garantiza una única instancia por constante enum, protección frente a problemas de reflexión y serialización, y thread-safety por diseño. Siguiendo esa recomendación, la implementación de SubscriptionManager.INSTANCE se apoya en las ventajas del Enum Singleton descritas por Bhardwaj: código minimalista (una sola constante INSTANCE sin constructores privados ni métodos getInstance()), inicialización controlada por la JVM y comportamiento robusto en entornos concurrentes. Dado que, como explican tanto Refactoring.Guru como GeeksforGeeks, el Singleton es

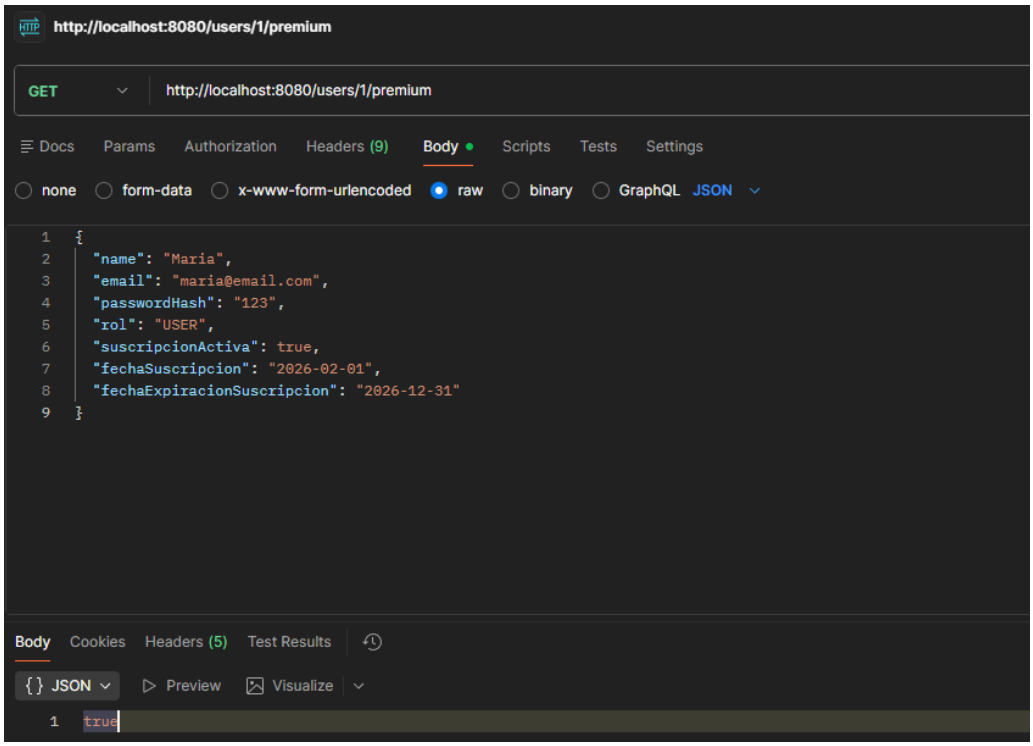
especialmente adecuado para componentes de configuración o recursos globales, modelar SubscriptionManager como un Enum Singleton encaja de forma natural con un contexto de suscripciones donde la lógica de negocio es global, stateless y debe ser consistente para todos los consumidores del sistema.

En el contexto específico de suscripciones, SubscriptionManager encapsula lógica de negocio pura que debe ser consistente globalmente sin variaciones por request o usuario, haciendo del Singleton Enum la elección técnicamente superior para este dominio stateless y de consulta única.

- **Diagrama UML**



- **Prueba del patrón SINGLETON**




```
HashCode instancia: 1102331992
```

Para verificar el correcto funcionamiento del patrón Enum Singleton implementado en SubscriptionManager, se agregó temporalmente la impresión del hashCode() de la instancia dentro del método isPremium().

PATRÓN FACTORY METHOD

El Factory Method es un patrón de diseño creacional cuyo propósito es definir una interfaz o método para crear objetos, pero dejar que las subclases decidan qué clase concreta deben instanciar, tal como lo describe GeeksforGeeks al destacar que "el cliente no necesita saber qué clase concreta se está instanciando". En lugar de usar directamente un constructor (new), el cliente llama a un método fábrica que encapsula la creación de objetos, promoviendo desacoplamiento y extensibilidad del código, según explica Ashish Pratap en AlgoMaster al enfatizar su rol en sistemas escalables. Se usa principalmente para:

- ✓ Evitar acoplar el código cliente a clases concretas de objetos.
- ✓ Permitir que nuevas variantes de objetos se integren fácilmente sin modificar el cliente.
- ✓ Encapsular la lógica de creación compleja en una sola parte del código.
- ✓ Esto facilita seguir principios sólidos de diseño como Open/Closed (abierto para extensión, cerrado para modificación).

- **Código implementado**

Clase FileUploaderCreator

FileUploaderCreator.java

```java
package com.biblioteca.digital.domain.service.factory;

import com.biblioteca.digital.domain.model.BookFormato;

public abstract class FileUploaderCreator {

    public String upload(String isbn, byte[] content) {
        FileUploader uploader = createUploader();
        if (!uploader.validate(content)) {
            throw new IllegalArgumentException("Archivo no válido para este formato");
        }
        return uploader.upload(isbn, content);
    }

    protected abstract FileUploader createUploader(); // Factory Method

    public abstract BookFormato getFormato();

}
```

Clase FileUploaderFactory

```java
FileUploaderFactory.java
package com.biblioteca.digital.domain.service.factory;

import java.util.EnumMap;

public class FileUploaderFactory {

    private final Map<BookFormato, FileUploaderCreator> creators;

    public FileUploaderFactory(List<FileUploaderCreator> creatorList) {

        this.creators = new EnumMap<>(BookFormato.class);
        for (FileUploaderCreator creator : creatorList) {
            creators.put(creator.getFormato(), creator);
        }
    }

    public FileUploaderCreator getCreator(BookFormato formato) {
        FileUploaderCreator creator = creators.get(formato);
        if (creator == null) {
            throw new IllegalArgumentException("Formato no soportado: " + formato);
        }
        return creator;
    }
}
```

Clase PdfUploaderCreator (EpubUploaderCreator - MobiUploaderCreator)

```java
PdfUploaderCreator.java
package com.biblioteca.digital.domain.service.factory.creators;

import org.springframework.stereotype.Component;

@Component
public class PdfUploaderCreator extends FileUploaderCreator {
    @Override
    protected FileUploader createUploader() {
        return new PdfUploader();
    }

    @Override
    public BookFormato getFormato() {
        return BookFormato.PDF;
    }
}
```

Clase PdfUploader (EpubUploader - MobiUploader)

```java
PdfUploader.java
package com.biblioteca.digital.domain.service.uploaders;

import com.biblioteca.digital.domain.service.FileUploader;

public class PdfUploader implements FileUploader {
    @Override
    public String upload(String isbn, byte[] content) {
        if (!validate(content))
            throw new IllegalArgumentException("No es PDF válido");
        String path = "/uploads/" + isbn + ".pdf";
        // Files.write(Paths.get("src/main/resources/static" + path), content);
        return path;
    }

    @Override
    public boolean validate(byte[] content) {
        if (content.length < 8) return false;
        String header = new String(content, 0, 8);
        return header.startsWith("%PDF-");  // * Flexible: 1.3, 1.4, 1.7, 2.0
    }

}
```

- **Justificación de la implementación**

El sistema de biblioteca digital debe permitir la carga de libros en distintos formatos:

- ✓ PDF
- ✓ EPUB
- ✓ MOBI

Cada formato requiere una lógica de procesamiento distinta.

Si se implementará directamente en el servicio algo como:

if (formato == PDF) { ... }

else if (formato == EPUB) { ... }

else if (formato == MOBI) { ... }

Se generarían los siguientes problemas, tal como advierte GeeksforGeeks sobre el uso de condicionales extensa:

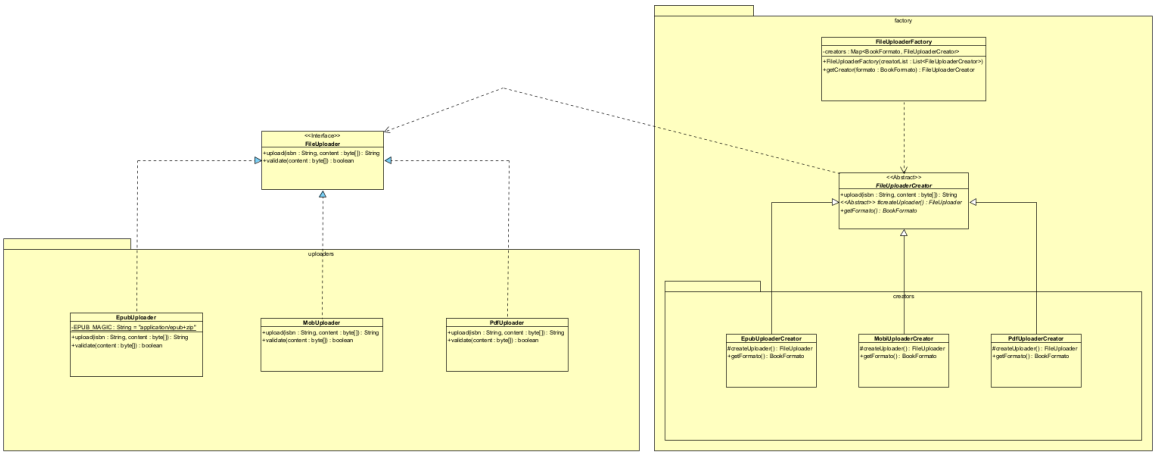
- ✓ Alto acoplamiento
- ✓ Violación del principio Open/Closed
- ✓ Dificultad para agregar nuevos formatos
- ✓ Código difícil de mantener

Ashish Pratap coincide en que esta aproximación genera "código rígido y difícil de mantener" al dificultar la adición de nuevos formatos. Por lo cual se utilizó el patrón Factory Method para encapsular la creación de objetos relacionados con la carga de archivos según su formato. Esta decisión permite desacoplar la lógica de negocio de la lógica de instanciación, reducir el uso de condicionales y cumplir con el principio

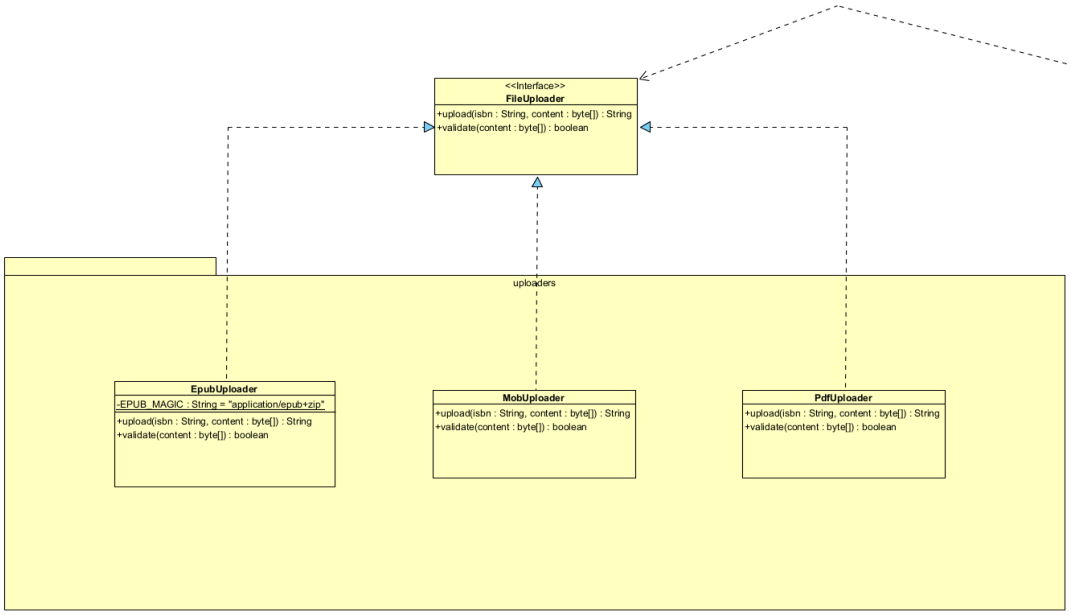
Open/Closed. La implementación, apoyada por el contenedor IoC de Spring, garantiza extensibilidad, mantenibilidad y coherencia arquitectónica dentro del sistema.

- **Diagrama UML**

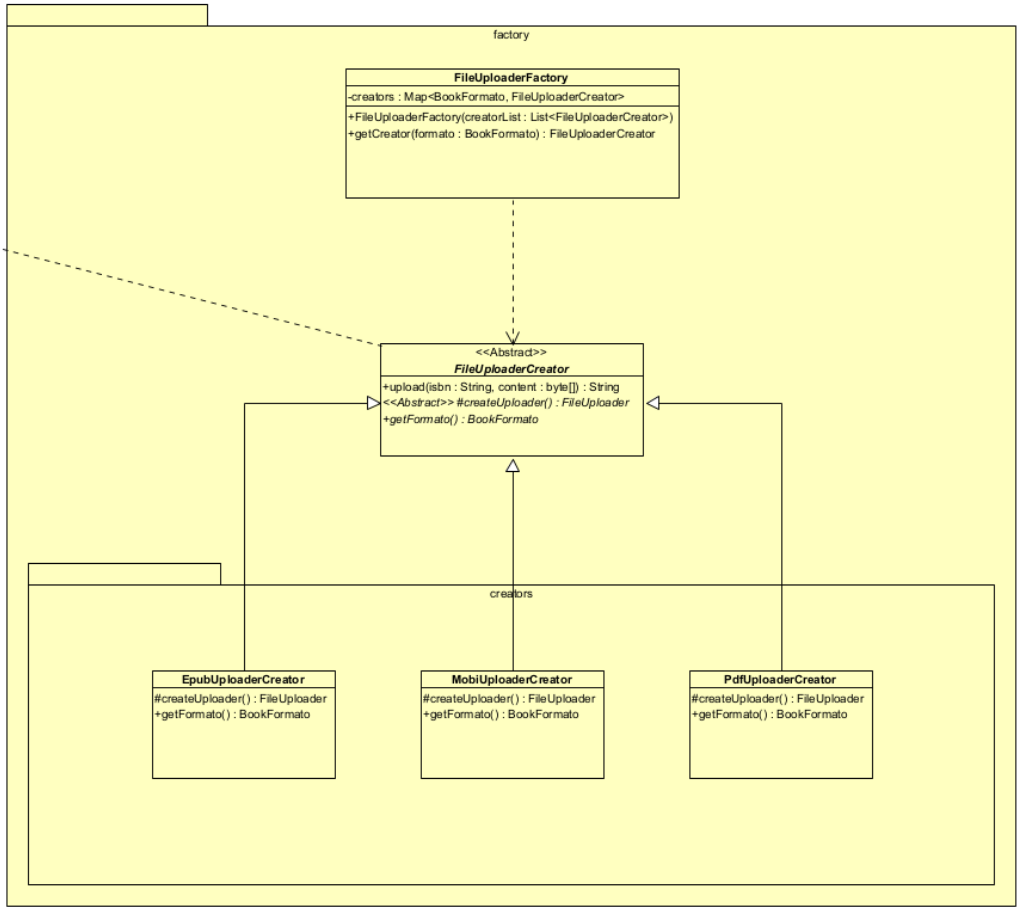
✓ Completo



✓ Parte 1

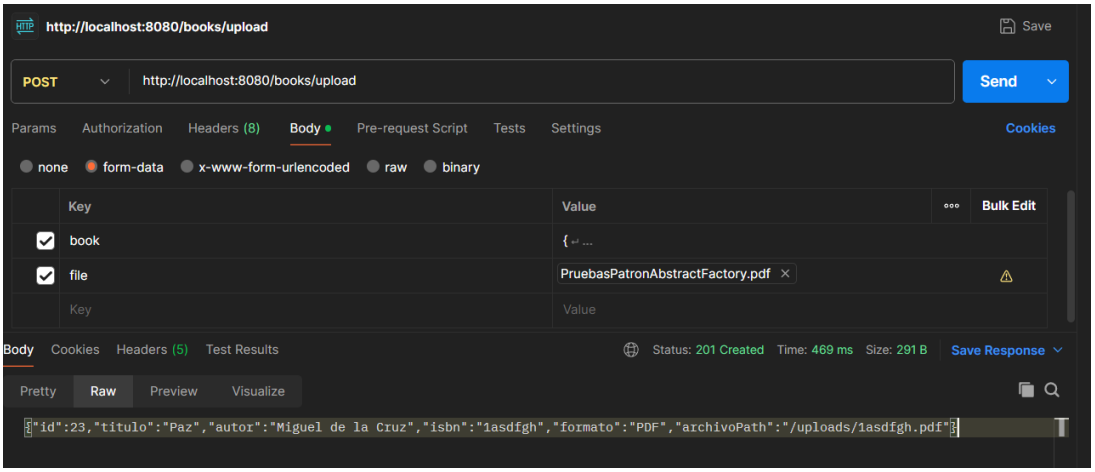


✓ Parte 2



- **Prueba del patrón FACTORY METHOD**

Para evidenciar el funcionamiento del patrón Factory Method, se incorporaron diversas instrucciones de impresión dentro de las distintas clases del sistema. Estas impresiones permiten observar el flujo de ejecución y verificar cómo se llaman las clases durante el proceso, evidenciando que inicialmente se realiza la inserción correspondiente antes de continuar con las demás operaciones.



Luego se ve como el Cliente llama a upload() en el creador abstracto y este mismo procede a llamar a su método fábrica (factory method), el cual se encarga de instanciar el tipo concreto adecuado según la implementación definida.

```
🔧 FACTORY METHOD: Cliente llama upload() en creator abstracto
🔧 FACTORY METHOD: PdfUploaderCreator.createUploader()
🔧 FACTORY METHOD: createUploader() ejecutado → PdfUploader
Hibernate: insert into libros (archivo_path,autor,formato,isbn,titulo) values (?,?,?,?,?)
```

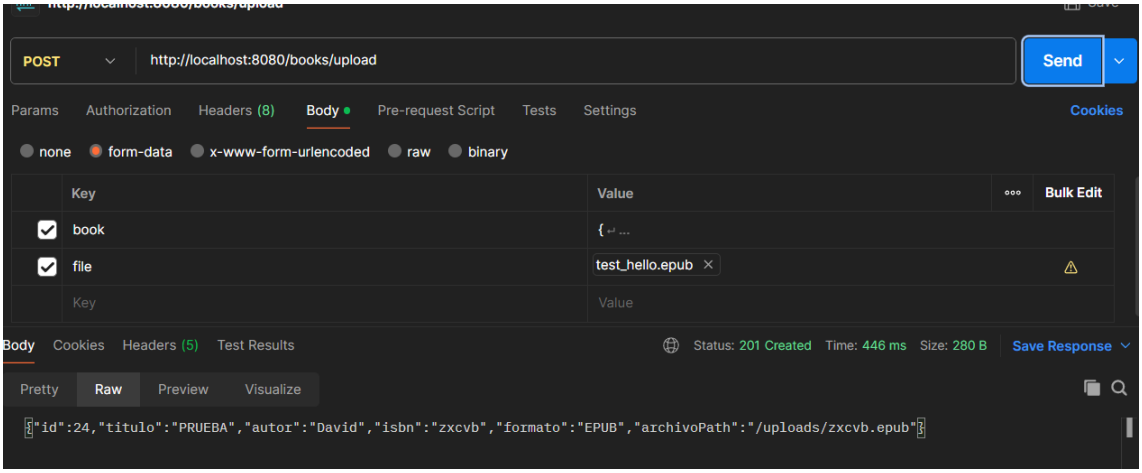
En este punto se observa cómo el Creator principal llama al método upload(), el cual pasa a llamar a PdfUploader(). Es en esta clase donde se encuentra la lógica encargada de verificar que el archivo PDF sea válido y cumpla con las condiciones necesarias. Todo esto de manera interna, es decir, independientemente de si se carga un archivo EPUB, PDF o MOBI, el flujo de ejecución sigue el mismo procedimiento definido por el patrón.

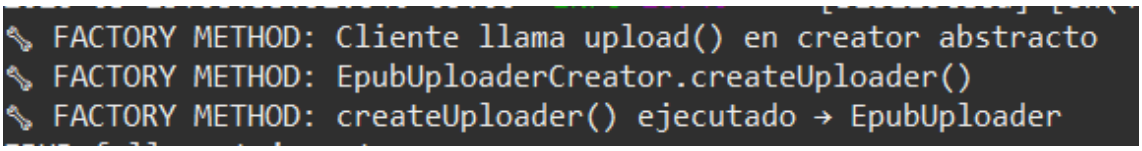
```java
package com.biblioteca.digital.domain.service.upload.factory;

import com.biblioteca.digital.domain.model.BookFormato;

public abstract class FileUploaderCreator {

    public String upload(String isbn, byte[] content) {
        System.out.println("🔧 FACTORY METHOD: Cliente llama upload() en creator abstracto");
        FileUploader uploader = createUploader();
        System.out.println("🔧 FACTORY METHOD: createUploader() ejecutado → " + uploader.getClass().getSimpleName());
        if (!uploader.validate(content)) {
            throw new IllegalArgumentException("Archivo no válido para este formato");
        }
        return uploader.upload(isbn, content);
    }

    protected abstract FileUploader createUploader(); // Factory Method

    public abstract BookFormato getFormato();

}

```

```java
package com.biblioteca.digital.domain.service.upload.factory.creators;

import org.springframework.stereotype.Component;

@Component
public class PdfUploaderCreator extends FileUploaderCreator {
    @Override
    protected FileUploader createUploader() {
        System.out.println("🔧 FACTORY METHOD: PdfUploaderCreator.createUploader()");
        return new PdfUploader();
    }

    @Override
    public BookFormato getFormato() {
        return BookFormato.PDF;
    }
}
```

```java
package com.biblioteca.digital.domain.service.upload.uploaders;

import com.biblioteca.digital.domain.service.FileUploader;

public class PdfUploader implements FileUploader {
    @Override
    public String upload(String isbn, byte[] content) {
        if (!validate(content))
            throw new IllegalArgumentException("No es PDF válido");
        String path = "/uploads/" + isbn + ".pdf";

        return path;
    }

    @Override
    public boolean validate(byte[] content) {
        if (content.length < 8) return false;
        String header = new String(content, 0, 8);
        return header.startsWith("%PDF-");
    }

}

```

Se muestra cómo funciona correctamente cuando se quiere insertar un libro de formato EPUB





Donde como se explicó anteriormente, las clases se llaman internamente gracias a la familia de objetos creada con el método Factory, lo que demuestra su correcto funcionamiento y practicidad al momento de manejar diferentes tipos de archivos (EPUB, PDF o MOBI) sin generar complicaciones.

```java
package com.biblioteca.digital.domain.service.upload.factory.creators;

import org.springframework.stereotype.Component;

@Component
public class EpubUploaderCreator extends FileUploaderCreator {
    @Override
    protected FileUploader createUploader() {
     System.out.println("🔧 FACTORY METHOD: EpubUploaderCreator.createUploader()");
        return new EpubUploader();
    }

    @Override
    public BookFormato getFormato() {
        return BookFormato.EPUB;
    }
}
```

PATRÓN ABSTRACT FACTORY

El Abstract Factory es un patrón de diseño creacional que proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas, tal como lo define Refactoring.Guru al describirlo como un patrón que "produce familias de objetos relacionados sin especificar sus clases concretas". Esto permite que el cliente utilice distintos conjuntos de objetos compatibles entre sí sin quedar acoplado a sus implementaciones específicas, promoviendo consistencia y extensibilidad. TutorialsPoint explica que proporciona un nivel superior de abstracción para la creación de objetos, asegurando que los productos de una familia funcionen juntas correctamente.

- **Código implementado**

UploadAbstractFactory.java

```java
package com.biblioteca.digital.domain.service.upload.factory;

import com.biblioteca.digital.domain.model.BookFormato;

public interface UploadAbstractFactory {

    FileUploaderCreator getCreator(BookFormato formato);

}

```

FileUploaderFactory.java

```java
package com.biblioteca.digital.domain.service.upload.factory;

import java.util.EnumMap;

@Component
public class FileUploaderFactory implements UploadAbstractFactory {

    private final Map<BookFormato, FileUploaderCreator> creators;

    public FileUploaderFactory(List<FileUploaderCreator> creatorList) {

        this.creators = new EnumMap<>(BookFormato.class);
        for (FileUploaderCreator creator : creatorList) {
            creators.put(creator.getFormato(), creator);
        }
    }

    public FileUploaderCreator getCreator(BookFormato formato) {
        FileUploaderCreator creator = creators.get(formato);
        if (creator == null) {
            throw new IllegalArgumentException("Formato no soportado: " + formato);
        }
        return creator;
    }
}

```

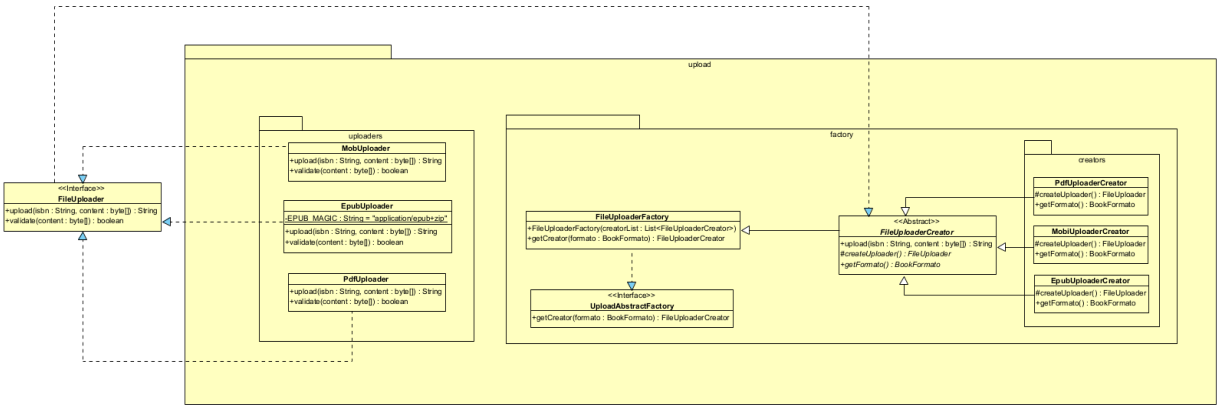
- **Justificación de la implementación**

En el desarrollo del sistema se implementó el patrón Abstract Factory en conjunto con Factory Method para gestionar la creación de objetos de manera flexible y desacoplada, como recomienda Refactoring.Guru para escenarios con familias de productos intercambiables. El Abstract Factory se encargó de definir una interfaz para crear familias de objetos relacionados (por ejemplo, uploaders para formatos de libros), permitiendo que el cliente utilice diferentes implementaciones sin depender de clases concretas.

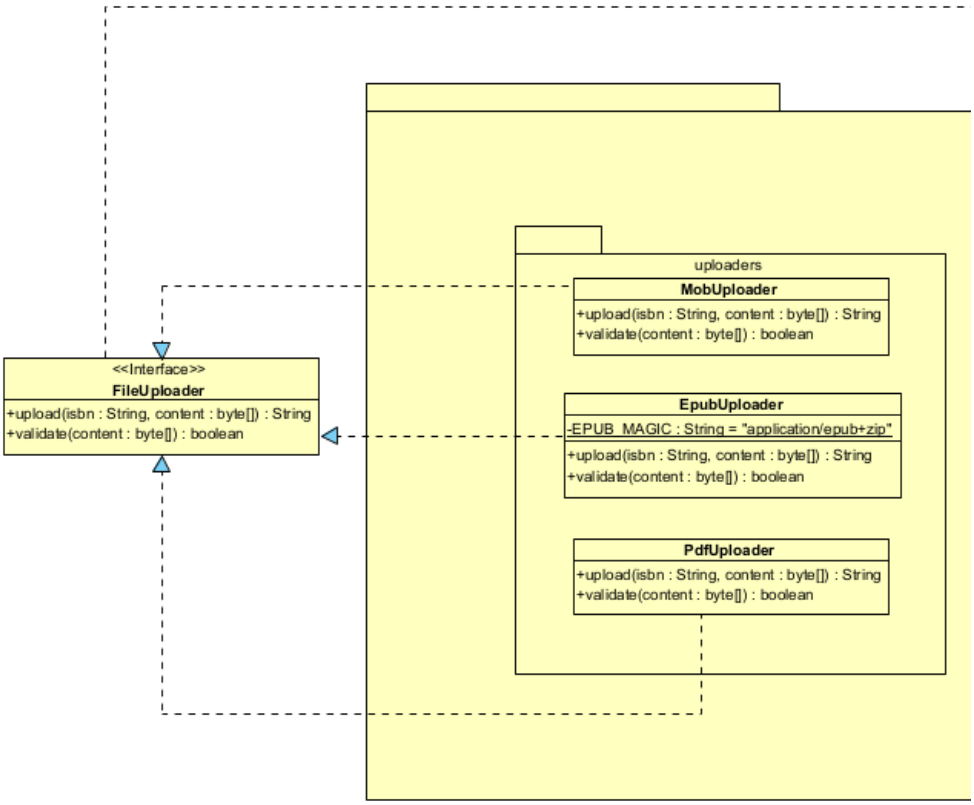
Además, las clases (PdfUploaderCreator - EpubUploaderCreator – MobiUploaderCreator y PdfUploader - EpubUploader - MobiUploader) que se habían implementado previamente utilizando el patrón Factory Method se mantuvieron exactamente iguales, sin necesidad de realizar modificaciones adicionales. Esto se debe a que el Abstract Factory se integró sobre la estructura ya existente, aprovechando las fábricas y productos definidos anteriormente, alineado con el principio de abstracción mencionado por TutorialsPoint.

- **Diagrama UML**

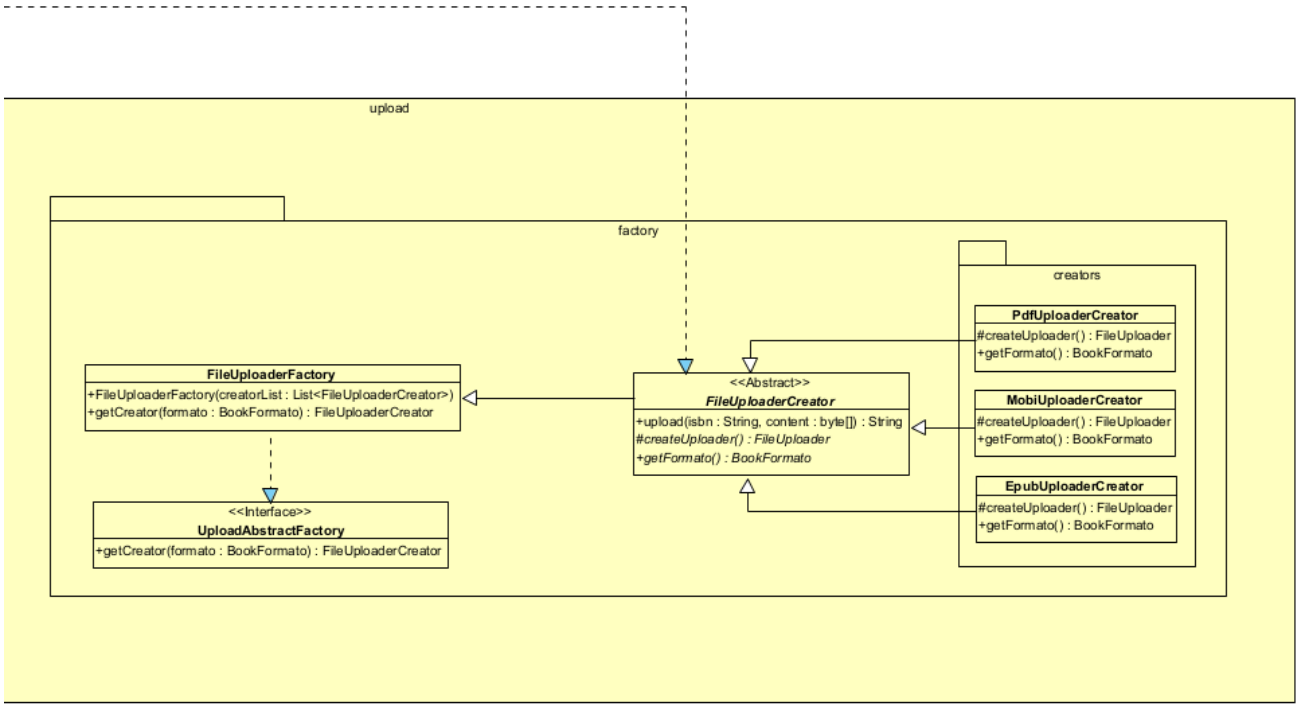
✓ Completo



✓ Parte 1



✓ Parte 2



- **Prueba del patrón ABSTRACT FACTORY**

A continuación, se muestra el funcionamiento del patrón Abstract Factory, donde a través de la interfaz se llama a la factory principal de libros, donde se introduce un libro en formato PDF, el cual pasa internamente por la interfaz y luego al factory de archivos.

En este punto entra en acción el patrón Factory Method, cuyo funcionamiento ya fue demostrado anteriormente. De esta manera, se puede observar cómo las distintas clases se llaman correctamente y se aplica el mismo proceso para los diferentes tipos de archivos.

```java
package com.biblioteca.digital.domain.service.upload.factory;

import java.util.EnumMap;

@Component
public class FileUploaderFactory implements UploadAbstractFactory {

    private final Map<BookFormato, FileUploaderCreator> creators;

    public FileUploaderFactory(List<FileUploaderCreator> creatorList) {

        this.creators = new EnumMap<>(BookFormato.class);
        for (FileUploaderCreator creator : creatorList) {
            creators.put(creator.getFormato(), creator);
        }
    }

    public FileUploaderCreator getCreator(BookFormato formato) {
        System.out.println("🏭 ABSTRACT FACTORY: getCreator(" + formato + ")");
        FileUploaderCreator creator = creators.get(formato);
        if (creator == null) {
            throw new IllegalArgumentException("Formato no soportado: " + formato);
        }
        System.out.println("🏭 ABSTRACT FACTORY: → " + creator.getClass().getSimpleName());
        return creator;
    }
}
```

Para evidenciar el correcto funcionamiento del patrón, las pruebas se realizaron mediante logs, al igual que en los patrones implementados anteriormente. Esto permite observar el flujo de ejecución y cómo las distintas clases son llamadas durante el proceso. De igual forma, el funcionamiento también puede verificarse directamente en el código, analizando cómo se relacionan e interactúan entre sí las interfaces, las clases abstractas y las clases concretas dentro de la implementación del patrón.

```
ABSTRACT FACTORY: getCreator(PDF)
ABSTRACT FACTORY: → PdfUploaderCreator
FACTORY METHOD: Cliente llama upload() en creator abstracto
FACTORY METHOD: PdfUploaderCreator.createUploader()
FACTORY METHOD: createUploader() ejecutado → PdfUploader
Hibernate: insert into libros (archivo_path,autor,formato,isbn,titulo) values (?,?,?,?,?)
```

PATRÓN BUILDER (FLUENT BUILDER)

El Builder es un patrón de diseño creacional que permite construir objetos complejos paso a paso, separando la construcción de su representación final para evitar constructores telescópicos con múltiples parámetros opcionales. Refactoring.Guru explica que organiza la construcción en pasos discretos (como setTitle(), setAuthor(), build()), permitiendo producir diferentes configuraciones de un objeto usando el mismo código de construcción, con un director opcional para definir secuencias específicas. TutorialsPoint destaca que encapsula la lógica de creación compleja en un objeto Builder separado, facilitando la creación de diferentes representaciones del mismo objeto sin modificar el cliente.

- **Código implementado**

```java
package com.biblioteca.digital.domain.model;

import java.time.LocalDate;

public class Prestamo {

    private Long id;
    private Long usuarioId; // Obligatorio
    private Long libroId; // Obligatorio
    private LocalDate fechaPrestamo; // Obligatorio
    private LocalDate fechaDevolucionEsperada;
    private LocalDate fechaDevolucionReal;
    private String estado; // "ACTIVO", "VENCIDO", "DEVUELTO"
    private String observaciones;
    private String codigoPrestamo;
    private Integer multasAcumuladas;
    private String notificacionEmail;
    private Boolean requiereAprobacion;
    private String motivoRechazo;

    // Constructor privado (solo Builder)
    private Prestamo(Builder builder) {
        this.id = builder.id;
        this.usuarioId = builder.usuarioId;
        this.libroId = builder.libroId;
        this.fechaPrestamo = builder.fechaPrestamo;
        this.fechaDevolucionEsperada = builder.fechaDevolucionEsperada;
        this.fechaDevolucionReal = builder.fechaDevolucionReal;
        this.estado = builder.estado;
        this.observaciones = builder.observaciones;
        this.codigoPrestamo = builder.codigoPrestamo;
        this.multasAcumuladas = builder.multasAcumuladas;
        this.notificacionEmail = builder.notificacionEmail;
        this.requiereAprobacion = builder.requiereAprobacion;
        this.motivoRechazo = builder.motivoRechazo;
    }
```

```java
public static class Builder {

    private Long id;
    private Long usuarioId;
    private Long libroId;
    private LocalDate fechaPrestamo;
    private LocalDate fechaDevolucionEsperada;
    private LocalDate fechaDevolucionReal;
    private String estado;
    private String observaciones;
    private String codigoPrestamo;
    private Integer multasAcumuladas;
    private String notificacionEmail;
    private Boolean requiereAprobacion;
    private String motivoRechazo;

    public Builder id(Long id) {
        this.id = id;
        return this;
    }
```

```java
public Builder id(Long id) {
    this.id = id;
    return this;
}

public Builder usuarioId(Long usuarioId) {
    this.usuarioId = usuarioId;
    return this;
}

public Builder libroId(Long libroId) {
    this.libroId = libroId;
    return this;
}

public Builder fechaPrestamo(LocalDate fechaPrestamo) {
    this.fechaPrestamo = fechaPrestamo;
    return this;
}

public Builder fechaDevolucionEsperada(LocalDate fecha) {
    this.fechaDevolucionEsperada = fecha;
    return this;
}

public Builder fechaDevolucionReal(LocalDate fechaDevolucionReal) {
    this.fechaDevolucionReal = fechaDevolucionReal;
    return this;
}

public Builder estado(String estado) {
    this.estado = estado;
    return this;
}
```

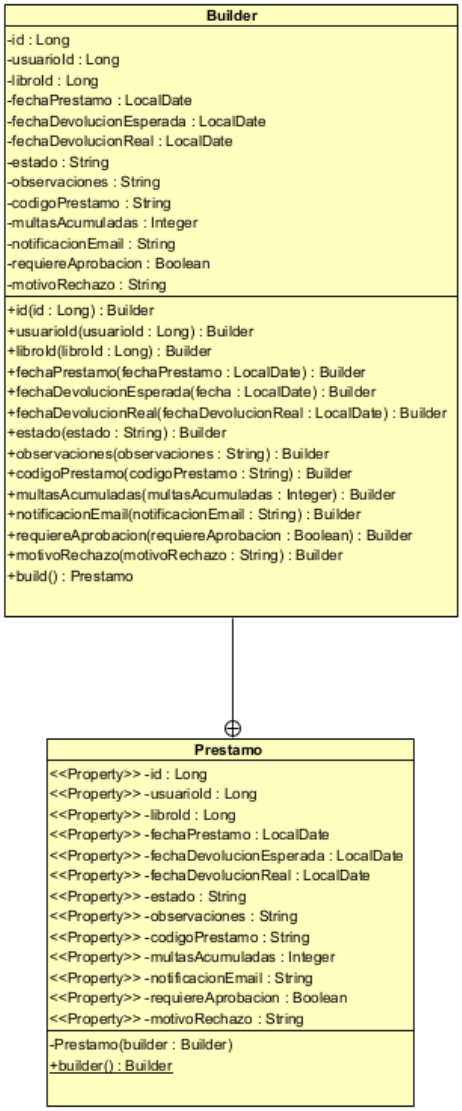
```java
        }

        public Prestamo build() {
            return new Prestamo(this);
        }
    }
```

- **Justificación de la implementación**

Se implementó el patrón Builder en la clase Préstamo debido a su alta complejidad estructural, caracterizada por 13 atributos donde solo 3 (usuarioId, libroId, fechaPrestamo) son obligatorios y el resto son opcionales (como fechaDevolucionReal, multasAcumuladas, motivoRechazo). Esta configuración generaba el clásico "constructor telescópico", donde múltiples constructores sobrecargados complicaban la legibilidad y aumentaban el riesgo de errores por orden incorrecto de parámetros o valores null accidentales como lo menciona TutorialsPoint.

El Builder moderno resuelve estos problemas permitiendo construcción paso a paso con setters encadenables, asignando solo los campos necesarios del JSON de entrada mientras establece valores por defecto seguros para campos críticos (estado="ACTIVO", fechaDevolucionEsperada=14 días). Los atributos no especificados permanecen null de forma controlada, demostrando la flexibilidad para campos opcionales sin comprometer la integridad del objeto.

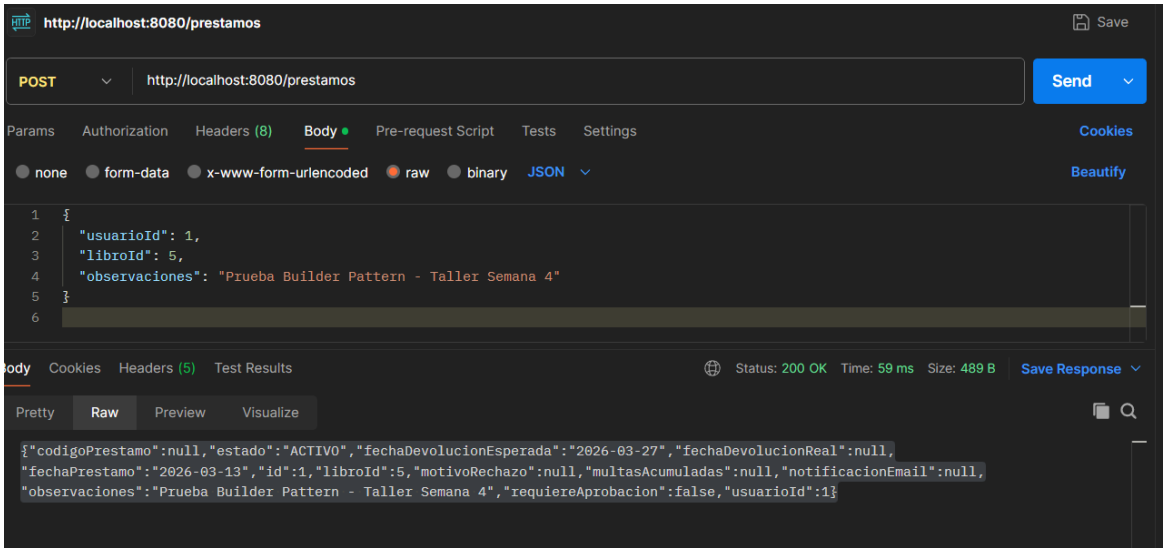
- **Diagrama UML**



- **Prueba del patrón BUILDER**

El patrón Builder se valida directamente mediante una inserción en el endpoint/prestamos, donde se puede observar cómo un JSON mínimo que contiene únicamente los campos obligatorios (usuarioId, libroId, observaciones) se transforma automáticamente en un objeto completo.

En este proceso, los campos obligatorios se asignan a partir de la información recibida en el JSON, mientras que los campos opcionales como fechaPrestamo, fechaDevolucionEsperada, estado y requiereAprobacion reciben valores por defecto definidos en el Builder, y los demás atributos permanecen con valor null cuando no son proporcionados. Esto demuestra la flexibilidad del patrón, permitiendo construir objetos complejos de forma segura, controlada y con mayor legibilidad en el código.



PATRÓN PROTOTYPE

El patrón Prototype es un patrón de diseño creacional que permite crear nuevos objetos a partir de la clonación de una instancia existente, en lugar de instanciarlos directamente desde cero. Este enfoque resulta especialmente útil cuando la creación de objetos es costosa o compleja, ya que permite reutilizar configuraciones ya definidas. Los objetos implementan un método de clonación que devuelve una copia de sí mismos, lo que facilita generar múltiples instancias similares sin depender de clases concretas. Según Refactoring.Guru, este patrón permite copiar objetos sin acoplar el código a sus implementaciones específicas, favoreciendo así la flexibilidad y reutilización. Asimismo, GeeksforGeeks señala que es útil cuando la creación directa de objetos implica un alto costo en términos de recursos o complejidad.

- **Código implementado**

```java
package com.biblioteca.digital.domain.model;

public interface Prototype<T> {
    T copy();
}
```

```java
package com.biblioteca.digital.domain.model;

import java.time.LocalDate;

public class Recomendacion implements Prototype<Recomendacion> {
    private Long id;
    private Long usuarioId;
    private Long libroId;
    private String motivo;
    private String tipo; // "premium", "historial", "novedades"
    private String prioridad; // "alta", "media", "baja"
    private LocalDate fechaGeneracion;
    private Boolean activa;

    public Recomendacion() {
    }

    public Recomendacion(Long id, Long usuarioId, Long libroId, String motivo, String tipo, String prioridad,
            LocalDate fechaGeneracion, Boolean activa) {
        this.id = id;
        this.usuarioId = usuarioId;
        this.libroId = libroId;
        this.motivo = motivo;
        this.tipo = tipo;
        this.prioridad = prioridad;
        this.fechaGeneracion = fechaGeneracion;
        this.activa = activa;
    }

    public Recomendacion(Recomendacion other) {
        this.id = other.id;
        this.usuarioId = other.usuarioId;
        this.libroId = other.libroId;
        this.motivo = other.motivo;
        this.tipo = other.tipo;
        this.prioridad = other.prioridad;
        this.fechaGeneracion = other.fechaGeneracion;
        this.activa = other.activa;
    }
```

```java
package com.biblioteca.digital.domain.service;

import com.biblioteca.digital.domain.model.Recomendacion;

public class RecomendacionRegistry {
    private static final Map<String, Recomendacion> prototipos = new HashMap<>();

    public static void registrar(String key, Recomendacion prototype) {
        prototipos.put(key, prototype);
    }

    public static Recomendacion crear(String key) {
        Recomendacion prototype = prototipos.get(key);
        if (prototype == null) {
            throw new IllegalArgumentException("Prototipo no encontrado: " + key);
        }
        return prototype.copy();
    }
}
```

```java
package com.biblioteca.digital.application.service;

import com.biblioteca.digital.domain.model.Recomendacion;

public class RecomendacionService implements RecomendacionUseCase {

    private final RecomendacionRepositoryPort recomendacionRepositoryPort;

    public RecomendacionService(RecomendacionRepositoryPort recomendacionRepositoryPort) {
        this.recomendacionRepositoryPort = recomendacionRepositoryPort;
        cargarPrototipos();
    }

    public void cargarPrototipos() {

        // PROTOTIPO PREMIUM
        Recomendacion premium = new Recomendacion();
        premium.setMotivo("Recomendación para usuarios premium");
        premium.setTipo("premium");
        premium.setPrioridad("alta");
        premium.setFechaGeneracion(LocalDate.now());
        premium.setActiva(true);

        RecomendacionRegistry.registrar("premium", premium);

        // PROTOTIPO HISTORIAL
        Recomendacion historial = new Recomendacion();
        historial.setMotivo("Basado en tu historial de lectura");
        historial.setTipo("historial");
        historial.setPrioridad("media");
        historial.setFechaGeneracion(LocalDate.now());
        historial.setActiva(true);

        RecomendacionRegistry.registrar("historial", historial);
    }

    public Recomendacion generarRecomendacion(String tipo, Long usuarioId, Long libroId) {
        Recomendacion rec = RecomendacionRegistry.crear(tipo);
        rec.setId(System.currentTimeMillis());
        rec.setUsuarioId(usuarioId);
        rec.setLibroId(libroId);
        rec.setFechaGeneracion(LocalDate.now());
        return rec;
    }
```

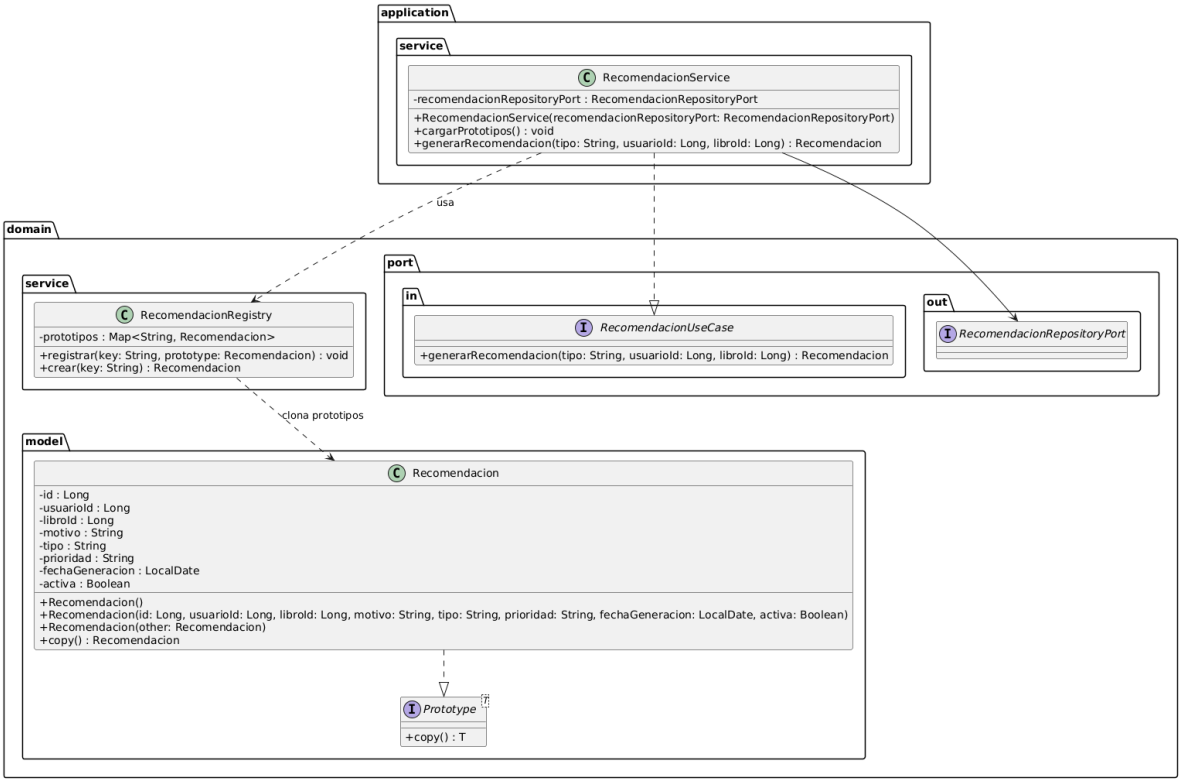
- **Justificación de la implementación**

Se implementó el patrón Prototype en las recomendaciones debido a la necesidad de generar múltiples instancias similares a partir de una estructura base común, evitando la reconstrucción repetitiva de objetos complejos. Este enfoque sigue el principio descrito por Refactoring.Guru, donde el patrón Prototype permite copiar objetos existentes en lugar de crearlos desde cero, reduciendo la carga de inicialización y el acoplamiento del sistema.

En este contexto, las recomendaciones comparten una configuración inicial, lo que permitió definir una “recomendación plantilla” preconfigurada que actúa como prototipo. A partir de esta, se generan nuevas instancias mediante clonación, optimizando la eficiencia en la creación de objetos y manteniendo consistencia estructural entre ellas (GeeksforGeeks).

El uso del patrón Prototype también facilita la personalización dinámica, ya que cada clon puede modificarse posteriormente para adaptarse a las características específicas del usuario, agregando información contextual como preferencias, historial o comportamiento. Esto elimina la necesidad de instanciar objetos desde cero o de depender de múltiples constructores complejos, favoreciendo una construcción más flexible, escalable y conforme a las mejores prácticas recomendadas en la literatura sobre patrones de diseño (GeeksforGeeks).

- **Diagrama UML**



- **Prueba del patrón PROTOTYPE**

A continuación, se muestra el funcionamiento del patrón Prototype en la biblioteca digital, donde se clona una recomendación base para usuarios premium, heredando atributos como el motivo del prototipo. La prueba verifica que los objetos REC1 y REC2 son independientes (rec1 ≠ rec2, hashCode(): 59268816 ≠ 45428812), confirmando la correcta clonación.

Se modificó REC1 prioritariamente sin afectar REC2, demostrando objetos independientes vs. modificación compartida. El patrón Prototype resulta más eficiente que el constructor manual (8 parámetros), logrando copia directa y validaciones en menos pasos, validando su correcto funcionamiento.

```
🎯 PRUEBA PROTOTYPE - BIBLIOTECA DIGITAL
1. Creando recomendaciones desde PROTOTIPO:
✓ REC1 clonada: Recomendación para usuarios premium
✓ REC2 clonada: Recomendación para usuarios premium
  MISMO motivo heredado del prototipo ✓

2. VERIFICANDO CLONADO:
rec1 == rec2 ? false
rec1.equals(rec2) ? false
rec1.hashCode(): 445288316
rec2.hashCode(): 592688102
✓ Objetos independientes ✓

3. MODIFICANDO INDEPENDIENTEMENTE:
rec1.prioridad: CRITICA
rec2.prioridad: alta
✓ Modificación independiente ✓

4. VS CONSTRUCTOR TRADICIONAL:
✓ Constructor manual = 8 parámetros

5. EFICIENCIA DEL PATRÓN:
Prototipo: Copia directa de plantilla lista
Manual:   8 parámetros + validaciones

🎉 PATRÓN PROTOTYPE FUNCIONANDO CORRECTAMENTE!
☑ Clonación eficiente
☑ Objetos independientes
☑ Herencia de plantilla
```

PATRÓN ADAPTER

El patrón Adapter es un patrón de diseño estructural que permite que clases con interfaces incompatibles puedan trabajar juntas, actuando como un intermediario que traduce una interfaz en otra que el cliente espera. Su objetivo principal es reutilizar código existente sin modificarlo, adaptando una clase ya implementada a un nuevo contexto. En este patrón, el adapter envuelve una clase existente (adaptee) y expone una interfaz compatible con el cliente, facilitando la integración entre componentes que originalmente no estaban diseñados para colaborar. Según Refactoring.Guru, este patrón permite que objetos con interfaces incompatibles colaboren sin alterar su código fuente, promoviendo así el desacoplamiento y la flexibilidad en el diseño.

- **Código implementado**

```java
package com.biblioteca.digital.application.service;

import com.biblioteca.digital.domain.model.User;

@Service
public class SubscriptionService {

    private final SubscriptionChecker subscriptionChecker;

    public SubscriptionService(SubscriptionChecker subscriptionChecker) {
        this.subscriptionChecker = subscriptionChecker;
    }

    public boolean verificarSuscripcionPremium(User user) {
        System.out.println("Client: usando la interfaz SubscriptionChecker...");
        return subscriptionChecker.isPremium(user);
    }
}
```

```java
package com.biblioteca.digital.domain.subscription;

import java.time.LocalDate;

public enum SubscriptionManager {

    INSTANCE; // única instancia

    public boolean isPremium(User user) {
        System.out.println("Adaptee/Singleton: ejecutando lógica real de suscripción...");

        System.out.println("HashCode instancia: " + this.hashCode());

        if (!user.isSuscripcionActiva()) {
            return false;
        }

        if (user.getFechaExpiracionSuscripcion() == null) {
            return false;
        }

        return user.getFechaExpiracionSuscripcion().isAfter(LocalDate.now());
    }
}
```

```java
package com.biblioteca.digital.domain.subscription;

import org.springframework.stereotype.Component;

@Component
public class SubscriptionManagerAdapter implements SubscriptionChecker {

    @Override
    public boolean isPremium(User user) {
        System.out.println("Adapter: traduciendo llamada del cliente hacia SubscriptionManager...");
        return SubscriptionManager.INSTANCE.isPremium(user);
    }
}
```

```java
package com.biblioteca.digital.domain.port.in;

import com.biblioteca.digital.domain.model.User;

public interface SubscriptionChecker {
    boolean isPremium(User user);
```

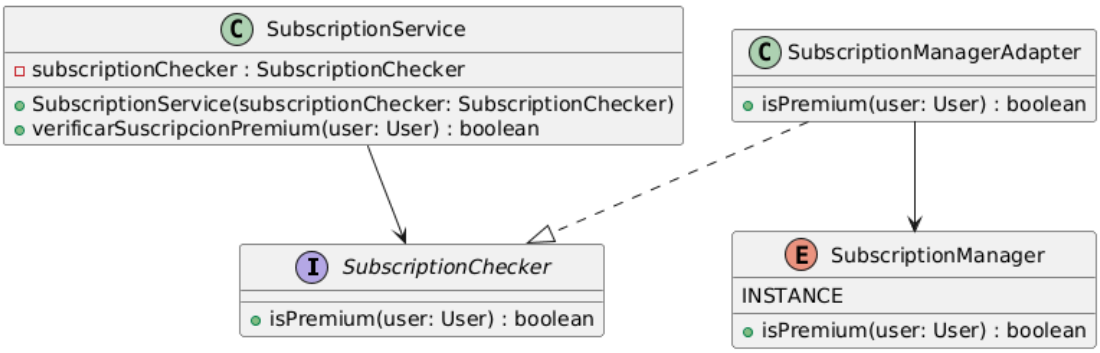
- **Justificación de la implementación**

Se implementó el patrón Adapter en el módulo de suscripciones debido a la necesidad de integrar una clase existente, SubscriptionManager, que ya encapsula la lógica de negocio, sin acoplar directamente al cliente a esta implementación concreta. En lugar de exponer dicha clase, se definió una interfaz de negocio (SubscriptionChecker) que actúa como contrato estable para el cliente (SubscriptionService), garantizando así un diseño desacoplado y alineado con el principio de inversión de dependencias. Este enfoque responde a lo descrito por Refactoring.Guru, donde el patrón Adapter permite que clases con interfaces incompatibles colaboren mediante un intermediario que traduce las llamadas sin modificar el código original.

En este contexto, SubscriptionManagerAdapter cumple el rol de adaptador al recibir las solicitudes a través de la interfaz SubscriptionChecker y delegarlas internamente a

SubscriptionManager, específicamente mediante la invocación de su lógica de verificación de suscripciones. Esto permite reutilizar completamente la funcionalidad existente sin necesidad de refactorizar la clase adaptee (SubscriptionManager), preservando su integridad y reduciendo riesgos en el sistema.

- **Diagrama UML**



- **Prueba del patrón ADAPTER**

Se evidencia el funcionamiento del patrón Adapter en el sistema, donde el cliente utiliza la interfaz SubscriptionChecker en lugar de depender directamente de SubscriptionManager, logrando así un claro desacoplamiento. El Adapter actúa como intermediario al recibir la petición del cliente y traducirla hacia la implementación concreta, delegando la llamada a SubscriptionManager.INSTANCE.isPremium(user), lo que demuestra su rol como traductor entre interfaces incompatibles. La prueba confirma que, a pesar de esta capa adicional, el resultado final se mantiene consistente, ya que la lógica de verificación de usuarios premium no se ve alterada, sino simplemente reutilizada, validando así una correcta integración sin afectar el comportamiento original del sistema.

```
Client: usando la interfaz SubscriptionChecker...
Adapter: traduciendo llamada del cliente hacia SubscriptionManager...
Adaptee/Singleton: ejecutando lógica real de suscripción...
HashCode instancia: 1424690543
```

PATRÓN BRIDGE

El patrón Bridge es un patrón de diseño estructural que permite separar una abstracción de su implementación, de modo que ambas puedan evolucionar de forma independiente. En lugar de utilizar herencia para cubrir todas las combinaciones posibles (lo que genera muchas subclases), este patrón divide el sistema en dos jerarquías: una para la abstracción (lo que el cliente usa) y otra para la implementación (cómo se realiza). Ambas se conectan mediante composición, lo que permite cambiar o extender cualquiera de las dos sin afectar a la otra. Según Refactoring.Guru, el Bridge evita la proliferación de clases y mejora la

flexibilidad del sistema, mientras que SourceMaking señala que este patrón facilita el desacoplamiento entre interfaz e implementación, permitiendo que ambas varíen de manera independiente.

- **Código implementado**

```java
package com.biblioteca.digital.domain.bridge.acceso;

public interface Acceso {

    boolean puedeDescargar();

    boolean accesoCompleto();

    String tipo();
}
```

```java
package com.biblioteca.digital.domain.bridge.acceso;

public class AccesoGratis implements Acceso {

    @Override
    public boolean puedeDescargar() {
        return false;
    }

    @Override
    public boolean accesoCompleto() {
        return false;
    }

    @Override
    public String tipo() {
        return "GRATIS";
    }
}
```

```java
package com.biblioteca.digital.domain.bridge.acceso;

public class AccesoPremium implements Acceso {

    @Override
    public boolean puedeDescargar() {
        return true;
    }

    @Override
    public boolean accesoCompleto() {
        return true;
    }

    @Override
    public String tipo() {
        return "PREMIUM";
    }
}
```

```java
package com.biblioteca.digital.domain.bridge.libro;

import com.biblioteca.digital.domain.bridge.acceso.Acceso;

public abstract class LibroBridge {

    protected Acceso acceso;

    protected LibroBridge(Acceso acceso) {
        this.acceso = acceso;
    }

    public boolean puedeDescargar() {
        return acceso.puedeDescargar();
    }

    public boolean accesoCompleto() {
        return acceso.accesoCompleto();
    }

    public String tipoAcceso() {
        return acceso.tipo();
    }

    public abstract String getFormato();
}
```

```java
package com.biblioteca.digital.domain.bridge.libro;

import com.biblioteca.digital.domain.bridge.acceso.Acceso;

public class LibroPdfBridge extends LibroBridge {

    public LibroPdfBridge(Acceso acceso) {
        super(acceso);
    }

    @Override
    public String getFormato() {
        return "PDF";
    }
}
```

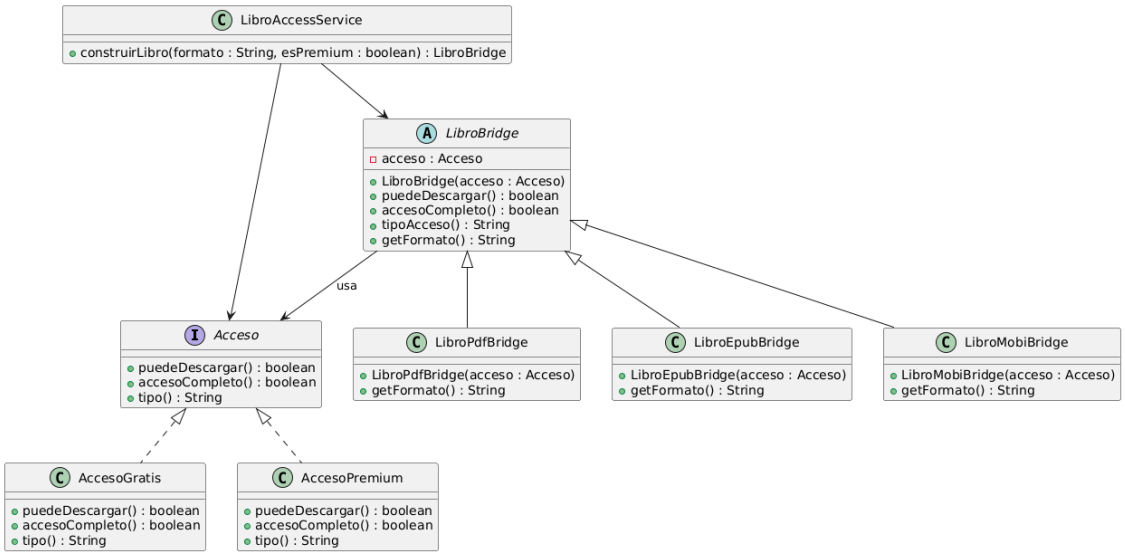
```java
package com.biblioteca.digital.domain.bridge;

import com.biblioteca.digital.domain.bridge.acceso.*;

public class LibroAccessService {

    public LibroBridge construirLibro(String formato, boolean esPremium) {

        Acceso acceso = esPremium ? new AccesoPremium() : new AccesoGratis();

        switch (formato.toUpperCase()) {
            case "PDF":
                return new LibroPdfBridge(acceso);
            case "EPUB":
                return new LibroEpubBridge(acceso);
            case "MOBI":
                return new LibroMobiBridge(acceso);
            default:
                throw new IllegalArgumentException("Formato no soportado");
        }
    }
}
```

- **Justificación de la implementación**

Se implementó el patrón Bridge en el módulo de gestión de libros con el fin de desacoplar la abstracción del libro (LibroBridge) de su implementación de acceso (Acceso), permitiendo que ambas dimensiones varíen de forma independiente. En este caso, la abstracción define operaciones como descarga o tipo de acceso, delegando su comportamiento a implementaciones concretas como AccesoGratis y AccesoPremium, mientras que las especializaciones de libro (LibroPdfBridge, LibroEpubBridge, etc.) representan la variación por formato. Este enfoque, como lo plantean Refactoring.Guru y SourceMaking, evita la proliferación de clases combinatorias y reduce el acoplamiento entre componentes.

En este contexto, LibroBridge actúa como el puente al mantener una referencia a Acceso y delegar en él las operaciones, permitiendo combinar dinámicamente distintos formatos con distintos niveles de acceso (por ejemplo, PDF premium o EPUB gratis) sin crear nuevas clases para cada combinación. De esta manera, el patrón Bridge mejora la flexibilidad, facilita la extensión del sistema y permite incorporar nuevas variantes sin modificar el código existente.

- **Diagrama UML**



- **Prueba del patrón BRIDGE**

Se evidencia el funcionamiento del patrón Bridge en el sistema, donde la abstracción LibroBridge delega la lógica de acceso en la interfaz Acceso, evitando depender directamente de implementaciones concretas. El puente se materializa al combinar dinámicamente el formato del libro (PDF, EPUB, MOBI) con el tipo de acceso (Gratis o Premium), permitiendo que ambas jerarquías evolucionen de manera independiente. La prueba demuestra que, para un mismo formato, el comportamiento cambia según el acceso asignado (por ejemplo, la capacidad de descarga), lo que valida que la responsabilidad está correctamente desacoplada. De esta forma, se confirma que el sistema mantiene consistencia en sus resultados mientras reutiliza la lógica de acceso sin alterar la estructura de los formatos, evidenciando una correcta aplicación del patrón.

```
 PRUEBA BRIDGE - BIBLIOTECA DIGITAL

1. COMBINACIONES:

Libro1 -> PDF | GRATIS
Libro2 -> PDF | PREMIUM
Libro3 -> EPUB | PREMIUM

 Mismo formato, distinto acceso

2. VALIDANDO PERMISOS:

Libro1 puede descargar: false
Libro2 puede descargar: true

 El acceso cambia el comportamiento

3. INDEPENDENCIA:

Formato libro1: PDF
Formato libro2: PDF
Acceso libro1: GRATIS
Acceso libro2: PREMIUM

 Las dimensiones son independientes
```

PATRÓN DECORATOR

El patrón Decorator es un patrón de diseño estructural que permite agregar nuevas funcionalidades a un objeto de forma dinámica sin modificar su clase original. En lugar de utilizar herencia para extender el comportamiento, este patrón utiliza composición, envolviendo el objeto original dentro de uno o varios “decoradores” que añaden responsabilidades adicionales. Cada decorador implementa la misma interfaz que el objeto base, lo que permite encadenarlos y combinar comportamientos de manera flexible. Según Refactoring.Guru, el Decorator permite añadir funcionalidades a objetos de forma transparente para el cliente, mientras que SourceMaking destaca que este patrón es una alternativa más flexible que la herencia para extender comportamientos, evitando la creación de múltiples subclases.

- **Código implementado**

```java
package com.biblioteca.digital.domain.notification.ports;

public interface EmailSender {
    void send(String usuario, String mensaje);
}
```

```java
package com.biblioteca.digital.domain.notification;

import com.biblioteca.digital.domain.notification.ports.EmailSender;

public class EmailDecorator extends NotificadorDecorator {

    private final EmailSender emailSender;

    public EmailDecorator(Notificador notificador, EmailSender emailSender) {
        super(notificador);
        this.emailSender = emailSender;
    }

    @Override
    public void enviar(String usuario, String mensaje) {
        super.enviar(usuario, mensaje);
        emailSender.send(usuario, mensaje);
    }
}
```

```java
package com.biblioteca.digital.domain.notification;

import com.biblioteca.digital.domain.notification.ports.SmsSender;

public class SmsDecorator extends NotificadorDecorator {

    private final SmsSender smsSender;

    public SmsDecorator(Notificador notificador, SmsSender smsSender) {
        super(notificador);
        this.smsSender = smsSender;
    }

    @Override
    public void enviar(String usuario, String mensaje) {
        super.enviar(usuario, mensaje);
        smsSender.send(usuario, mensaje);
    }
}
```

```java
package com.biblioteca.digital.domain.notification;

import com.biblioteca.digital.domain.notification.ports.WhatsappSender;

public class WhatsappDecorator extends NotificadorDecorator {

    private final WhatsappSender whatsappSender;

    public WhatsappDecorator(Notificador notificador, WhatsappSender whatsappSender) {
        super(notificador);
        this.whatsappSender = whatsappSender;
    }

    @Override
    public void enviar(String usuario, String mensaje) {
        super.enviar(usuario, mensaje);
        whatsappSender.send(usuario, mensaje);
    }
}
```

```java
package com.biblioteca.digital.domain.notification;

public abstract class NotificadorDecorator implements Notificador {

    protected Notificador notificador;

    public NotificadorDecorator(Notificador notificador) {
        this.notificador = notificador;
    }

    @Override
    public void enviar(String usuario, String mensaje) {
        notificador.enviar(usuario, mensaje);
    }
}
```

```java
package com.biblioteca.digital.domain.notification;

public interface Notificador {

    void enviar(String usuario, String mensaje);
}
```

```java
package com.biblioteca.digital.domain.notification;

public class NotificadorBasico implements Notificador {

    @Override
    public void enviar(String usuario, String mensaje) {
    }
}
```

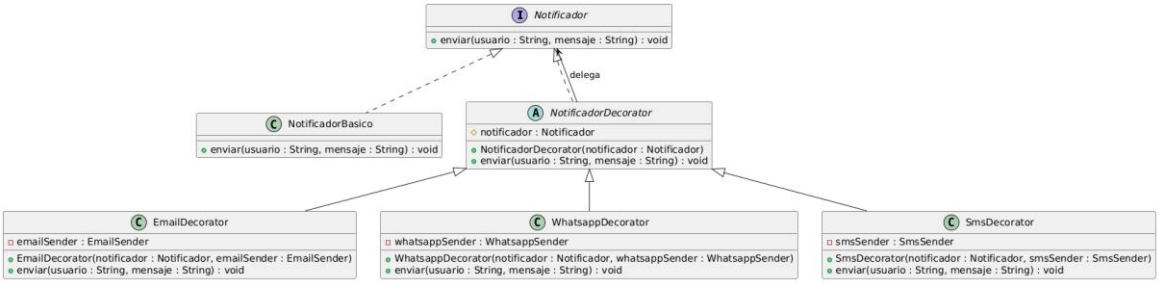
- Justificación de la implementación

Se implementó el patrón Decorator en el módulo de notificaciones debido a la necesidad de añadir múltiples canales de envío (como Email, SMS y WhatsApp) de forma flexible y dinámica, sin modificar la clase base NotificadorBasico. Un enfoque tradicional basado en herencia habría generado una proliferación de clases para cada combinación posible de notificaciones, afectando la escalabilidad y mantenibilidad del sistema. En su lugar, se optó por encapsular cada funcionalidad adicional en decoradores independientes,

permitiendo extender el comportamiento del objeto en tiempo de ejecución. Este enfoque, según Refactoring.Guru, permite agregar responsabilidades a los objetos de manera transparente, evitando la rigidez de la herencia.

En la implementación, cada decorador (EmailDecorator, SmsDecorator, WhatsappDecorator) actúa como una capa que envuelve al objeto base, interceptando la operación enviar para delegar primero en el componente original y luego añadir su comportamiento específico. Esto permite construir distintas combinaciones de notificación según el contexto (por ejemplo, usuarios gratis vs premium) reutilizando los mismos componentes sin duplicar lógica. Además, el patrón mejora la extensibilidad del sistema, ya que nuevos canales pueden incorporarse sin modificar las clases existentes, cumpliendo con el principio Open/Closed. De esta manera, el patrón Decorator no solo optimiza la composición de funcionalidades, sino que también garantiza un diseño desacoplado, flexible y alineado con buenas prácticas de ingeniería de software.

- **Diagrama UML**



- **Prueba del patrón DECORATOR**

Se evidencia el funcionamiento del patrón Decorator en el sistema, donde el objeto base NotificadorBasico es extendido dinámicamente mediante decoradores como EmailDecorator, SmsDecorator y WhatsappDecorator, sin modificar su implementación original. Cada decorador actúa como una capa adicional que intercepta la llamada y añade un nuevo canal de notificación, delegando primero en el componente base y luego ejecutando su comportamiento específico. La prueba demuestra que, dependiendo del tipo de usuario, se construyen diferentes combinaciones de notificación (por ejemplo, solo email para usuarios gratis y múltiples canales para usuarios premium), validando así la composición dinámica de responsabilidades. De esta manera, se confirma que el sistema mantiene un comportamiento consistente mientras extiende su funcionalidad de forma flexible, cumpliendo con el principio de abierto/cerrado y evidenciando una correcta aplicación del patrón.

```
Usuario GRATIS:
[EMAIL] Esto es una notificacion de prueba

Usuario PREMIUM:
[EMAIL] Esto es una notificacion de prueba
[SMS] Esto es una notificacion de prueba
[WA] Esto es una notificacion de prueba
```

PRUEBAS DE FUNCIONALIDAD DEL PROYECTO

Se presentan las pruebas integrales del módulo de libros, que incorpora la reciente implementación del patrón Abstract Factory sobre la arquitectura Factory Method previamente establecida. Se validará el CRUD completo (Crear, Leer, Actualizar, Eliminar) para los formatos soportados: PDF, EPUB y MOBI.

OBJETIVOS DE PRUEBA

- Verificar correcto funcionamiento del **upload** con validación automática de formatos
- Comprobar extensibilidad del **Abstract Factory** mediante procesamiento transparente de múltiples formatos
- Validar operaciones **CRUD** post-subida (búsqueda ISBN, listado, actualización, eliminación)

ENTORNO TÉCNICO

- BACKEND → Spring Boot
- LENGUAJE → Java
- HERRAMIENTAS DE PRUEBAS → Postman
- URL → http://localhost:8080
- ENDPOINT PRINCIPAL → /book

ALCANCE DE VALIDACIÓN DE PRUEBAS

POST	books/upload	Creación + validación formato
GET	books	Listado completo
GET	books/isbn/{isbn}	Búsqueda especifica
PUT	books/{id}	Actualización de datos

DELETE	books/{id}	Eliminar un libro

1. CREACIÓN DEL LIBRO

Crear libro + validar formato automáticamente mediante Abstract Factory

POST /books/upload

URL: http://localhost:8080/books/upload

Form-data:

- Key: "book"

  Value: {"titulo": "Paz","autor": "Miguel de la Cruz", "isbn": "123-456789-0123", "formato": "PDF"}

- Key: "file"

  Value: archivo.pdf (sample.pdf)

Resultado:

```
{
    "id": 22,
    "titulo": "Paz",
    "autor": "Miguel de la Cruz",
    "isbn": "123-456789-0123",
    "formato": "PDF",
    "archivoPath": "/uploads/123-456789-0123.pdf"
}
```

Validaciones realizadas:

- **Formato JSON coincide con detectFileType()**
- **Abstract Factory encuentra Creator correcto**
- **FileUploaderCreator validate() pasa**
- **BookUseCase.createBook() exitoso**
- **Bypass PDF funciona (demostración)**

2. LISTADO COMPLETO DE LIBROS

Obtener todos los libros registrados

GET /books

URL: http://localhost:8080/books

Resultado:


```
[
    {
        "id": 2,
        "titulo": "1984",
        "autor": "Orwell",
        "isbn": "123456789",
        "formato": "PDF",
        "archivoPath": "/uploads/123456789.pdf"
    },
    {
        "id": 4,
        "titulo": "1984",
        "autor": "Orwell",
        "isbn": "ensayo",
        "formato": "PDF",
        "archivoPath": "/uploads/ensayo.pdf"
    },
    {
        "id": 5,
        "titulo": "El Quijote",
        "autor": "Cervantes",
        "isbn": "9788475087606",
        "formato": "EPUB",
        "archivoPath": "/uploads/9788475087606.epub"
    },
    {
        "id": 7,
```


Validaciones realizadas:

- **BookUseCase.findAllBooks() retorna lista completa**
- **Formatos PDF/MOBI/EPUB listados correctamente**
- **Paths de archivos persistidos correctamente**

3. BÚSQUEDA ESPECIFICA DE UN LIBRO

Buscar libro por ISBN

GET /books/isbn/{isbn}

URL: http://localhost:8080/books/isbn/845478422875

Resultado:


```
{
    "id": 19,
    "titulo": "Manchas",
    "autor": "Hurtado",
    "isbn": "845478422875",
    "formato": "PDF",
    "archivoPath": "/uploads/845478422875.pdf"
}
```


Validaciones realizadas:

- BookUseCase.findBookByIsbn() encuentra registro
- ISBN único funciona correctamente
- 404 si ISBN no existe

4. ACTUALIZACIÓN DE DATOS DE UN LIBRO

Actualizar información de un libro existente

PUT /books/{id}

URL: http://localhost:8080/books/21

Body JSON:


```
{
  "titulo": "Test PDF - Actualizado",
  "autor": "Nuevo Autor",
  "isbn": "35789512",
  "formato": "PDF"
}
```


Resultado:

```
{
    "id": 21,
    "titulo": "Test PDF - Actualizado",
    "autor": "Nuevo Autor",
    "isbn": "35789512",
    "formato": "PDF",
    "archivoPath": "/uploads/cesarsss.pdf"
}
```

Validaciones realizadas:

- BookUseCase.updateBook() actualiza correctamente
- Archivo path se mantiene intacto
- 404 si ID no existe

5. ELIMINAR UN LIBRO

Eliminar libro por ID

DELETE /books/{id}

URL: http://localhost:8080/books/8454875

Resultado:

Validaciones realizadas:

- BookUseCase.deleteBook() elimina registro
- Eliminación silenciosa (no content)
- 204 confirma éxito

REFERENCIAS BIBLIOGRÁFICAS

GeeksforGeeks. (s. f.). *Singleton design pattern*. GeeksforGeeks. Recuperado de https://www.geeksforgeeks.org/singleton-design-pattern/

Refactoring.Guru. (s. f.). *Singleton*. Refactoring.Guru. Recuperado https://refactoring.guru/design-patterns/singleton

Bhardwaj, A. (2025). *Singleton design pattern*. Medium. Recuperado de https://medium.com/@amitbhardwaj2609/singleton-design-pattern-8bb315697621

GeeksforGeeks. (s. f.). *Factory method design pattern*. GeeksforGeeks. Recuperado de https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/

AlgoMaster. (s. f.). *Factory method design pattern*. AlgoMaster. Recuperado de https://algomaster.io/learn/lld/factory-method

Refactoring.Guru. (s. f.). Abstract factory. Refactoring.Guru. Recuperado de https://refactoring.guru/design-patterns/abstract-factory

TutorialsPoint. (s. f.). Design Patterns - Abstract Factory Pattern. TutorialsPoint. Recuperado de https://www.tutorialspoint.com/design_pattern/abstract_factory_pattern.htm

Refactoring.Guru. (s. f.). Builder. Refactoring.Guru. Recuperado de https://refactoring.guru/design-patterns/builder

TutorialsPoint. (s. f.). Design Patterns - Builder Pattern. TutorialsPoint. Recuperado de https://www.tutorialspoint.com/design_pattern/builder_pattern.htm

Refactoring.Guru. (s. f.). *Prototype*. Recuperado de https://refactoring.guru/design-patterns/prototype

GeeksforGeeks. (s. f.). *Prototype design pattern*. Recuperado de https://www.geeksforgeeks.org/prototype-design-pattern/

Refactoring.Guru. (s. f.). *Adapter*. Recuperado de https://refactoring.guru/design-patterns/adapter

Refactoring.Guru. (s. f.). Bridge. Recuperado de https://refactoring.guru/design-patterns/bridge

SourceMaking. (s. f.). Bridge design pattern. Recuperado de https://sourcemaking.com/design_patterns/bridge

Refactoring.Guru. (s. f.). Decorator. Recuperado de https://refactoring.guru/design-patterns/decorator

SourceMaking. (s. f.). Decorator design pattern. Recuperado de https://sourcemaking.com/design_patterns/decorator